МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

КУРСОВОЙ ПРОЕКТ

по дисциплине «Технологии автоматизации процесса разработки программного обеспечения»

Студентка гр. 1346 _____ Тараненко Э. А.

Преподаватель _____ Бутенко Д. С

Санкт-Петербург

2026

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ

Студент: Тараненко Элина Алексеевна.

Группа: 1346.

Тема проекта: реализация Docker-конфигурации из двух контейнеров: app с веб-приложением и tester с тестами для проверки приложения.

Исходные данные: вариант 80. Необходимо реализовать проверку HTML-шаблонов бьютификатором, статический анализ по собственному критерию, интеграционный тест загрузки файла, внешний SSH-доступ к контейнерам app и tester по паролю, логирование каждого этапа тестирования в docker logs и в общие файлы stdout/stderr, передачу порта веб-сервера через .env и ограничение числа CPU.

Содержание пояснительной записки: «Аннотация», «Summary», «Содержание», «Введение», «Постановка задачи», «Описание Dockerfile», «Описание скриптов запуска тестов», «Описание Docker-compose конфигурации», «Заключение», «Приложение А. Исходный код проекта», «Список использованных источников».

Предполагаемый объем пояснительной записки: не менее 15 страниц.

Дата выдачи задания: 21.03.2026.

Дата сдачи реферата: 29.04.2026.

Дата защиты реферата: 29.04.2026.

Студентка: _____ Тараненко Э. А.

Преподаватель: _____ Бутенко Д. С.

АННОТАЦИЯ

В рамках проекта была создана контейнеризированная среда, предназначенная для развертывания веб-приложения и выполнения автоматических тестов. Проект состоит из двух контейнеров: app, в котором запускается демонстрационное Flask-приложение, и tester, предназначенного для выполнения тестового pipeline.

В работе выполнена настройка Dockerfile для каждого контейнера, описана конфигурация Docker Compose, реализован SSH-доступ по паролю, организована передача параметров через .env и задано ограничение ресурсов. Для проверки приложения подготовлены тесты форматирования HTML, статического анализа с пользовательским правилом pylint и интеграционный тест загрузки файла.

Результаты тестирования выводятся в стандартные потоки контейнера и дополнительно сохраняются в файлы логов с timestamp, что позволяет анализировать выполнение каждого этапа как через docker logs, так и по сохраненным файлам.

SUMMARY

This course project presents a containerized infrastructure for running a web application and automated tests. The project includes two Docker containers: app for the Flask application and tester for the testing pipeline. Docker Compose is used to configure service interaction, port forwarding, volumes, environment variables, SSH access and CPU limits.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
ПОСТАНОВКА ЗАДАЧИ 6
1 ОПИСАНИЕ DOCKERFILE 7
1.1 Dockerfile_app 7
1.2 Dockerfile_tester 8
2 ОПИСАНИЕ СКРИПТОВ ЗАПУСКА ТЕСТОВ 10
2.1. Общий pipeline тестирования 10
2.2. Проверка HTML-шаблонов 11
2.3. Пользовательское правило pylint 11
2.4. Интеграционный тест загрузки файла 12
3 ОПИСАНИЕ DOCKER-COMPOSE КОНФИГУРАЦИИ 13
ЗАКЛЮЧЕНИЕ 15
ПРИЛОЖЕНИЕ А. Исходный код проекта 17
ПРИЛОЖЕНИЕ Б. Ручной запуск тестов в контейнере tester 25

ВВЕДЕНИЕ

Контейнеризация является одним из основных подходов к развертыванию современных программных систем. Docker позволяет описывать окружение приложения в виде воспроизводимых образов, а Docker Compose используется для совместного запуска нескольких связанных сервисов.

В данной работе рассматривается разработка Docker Compose-конфигурации, состоящей из двух сервисов: app и tester. Сервис app предназначен для запуска демонстрационного веб-приложения, а сервис tester используется для автоматизированной проверки приложения.

Особое внимание уделяется организации тестового pipeline, настройке сетевого взаимодействия между контейнерами, SSH-доступу, передаче параметров через переменные окружения, ограничению ресурсов и сохранению логов выполнения тестов.

ПОСТАНОВКА ЗАДАЧИ

Целью работы является разработка контейнеризированной конфигурации, позволяющей запускать веб-приложение и выполнять автоматизированные проверки в изолированном тестовом окружении. Для решения задачи используется Docker и Docker Compose.

Проект должен включать два контейнера. Контейнер app отвечает за запуск веб-приложения, а контейнер tester содержит инструменты и скрипты для проверки приложения. Между контейнерами используется стандартная Docker-сеть, в которой сервис tester обращается к приложению по адресу http://app:5000.

В соответствии с вариантом 80 необходимо выполнить условия, представленные в таблице 1.

Таблица 1 – Исходные данные

Категория	Требование варианта 80	Реализация в проекте
Организация тестов	Причесывание HTML бьютификатором	Скрипт test_html_format.py запускает css-html-prettify для HTML-шаблонов приложения.
Организация тестов	Создание своего критерия статического анализа	Скрипт test_custom_rule.py подключает пользовательский checker pylint и запрещает имя переменной elina.
Организация тестов	Интеграционный тест загрузки файла	Скрипт test_upload.py отправляет файл на endpoint /upload и проверяет JSON-ответ.
Docker	SSH-доступ в app и tester по паролю	В обоих Dockerfile создается пользователь и задается пароль, передаваемые через build args.
Docker	Логи tester в docker logs и в файлы stdout/stderr	run_tests.py пишет сообщения в stdout/stderr, /proc/1/fd/1, /proc/1/fd/2 и файлы tester/logs.
Docker Compose	Порт веб-сервера через .env	APP_PORT задает внешний порт сервиса app.
Docker Compose	Ограничение CPU	Параметр CPUS применяется через cpus в docker-compose.yml.

1 ОПИСАНИЕ DOCKERFILE

В проекте используются два Dockerfile: Dockerfile_app для контейнера с веб-приложением и Dockerfile_tester для контейнера тестирования. Разделение образов позволяет отделить runtime приложения от инструментов проверки, снизить связность компонентов и сделать тестовую среду независимой.

Оба образа основаны на ubuntu:22.04. Такой выбор соответствует требованиям задания и обеспечивает предсказуемую среду с доступными версиями системных пакетов. Установка apt-пакетов выполняется в одном RUN-слое, после чего очищается кэш apt, что уменьшает размер итогового образа.

1.1 Dockerfile_app

Dockerfile_app формирует контейнер app, предназначенный для запуска Flask-приложения. В образ устанавливаются git, Python, pip, SSH-сервер, patch и ca-certificates. Все системные зависимости указаны с конкретными версиями, что повышает воспроизводимость сборки.

Основные элементы Dockerfile_app и их назначение приведены в таблице 1.1.

Таблица 1.1 – Основные элементы Dockerfile_app

Элемент	Назначение
ARG SSH_USER, SSH_PASSWORD	Параметры пользователя и пароля для SSH-доступа, передаваемые из docker-compose.yml.
ENV APP_DIR=/app	Единая переменная для рабочей директории приложения.
apt-get install	Установка Python, pip, git, OpenSSH и вспомогательных утилит.

Продолжение таблицы 1.1

useradd и chpasswd	Создание пользователя внутри контейнера и назначение пароля.
git clone moevm/devops-examples	Получение исходников демонстрационного приложения EXAMPLE_APP во время сборки.
sed main.py	Настройка запуска Flask на 0.0.0.0:5000, чтобы приложение было доступно из контейнерной сети.
pip3 install	Установка Python-зависимостей приложения с фиксированными версиями.
CMD service ssh start && exec python3 main.py	Запуск SSH-сервера и передача управления процессу Flask-приложения.

Фрагмент настройки приложения:

```
RUN git clone https://github.com/moevm/devops-examples.git /tmp/devops-examples \
    && mkdir -p "${APP_DIR}" \
    && cp -r /tmp/devops-examples/EXAMPLE_APP/* "${APP_DIR}" \
    && rm -rf /tmp/devops-examples


RUN sed -i "s/app.run(debug = True)/app.run(host='0.0.0.0', port=5000, debug=True)/"
"${APP_DIR}/main.py"
```

Команда запуска использует exec перед python3 main.py. Это нужно для того, чтобы основной процесс приложения стал главным процессом контейнера после запуска SSH-сервера. Такой подход упрощает обработку сигналов завершения и соответствует требованию о корректном корне дерева процессов.

Листинг кода представлен в приложении А.

1.2 Dockerfile_tester

Dockerfile_tester формирует тестовое окружение. В нем устанавливаются Python, pip, SSH-сервер, requests, css-html-prettify и pylint. Контейнер tester не содержит исходники приложения внутри образа: доступ к ним предоставляется

через volume /app в режиме read-only. Такой подход позволяет тестировать именно тот код, который находится в контейнере app.

Основные элементы Dockerfile_tester приведены в таблице 1.2.

Таблица 1.2 – Основные элементы Dockerfile_tester

Элемент	Назначение
ENV TESTER_DIR=/tester	Рабочая директория для тестовых скриптов и логов.
requests	Библиотека для интеграционного HTTP-теста загрузки файла.
css-html-prettify	Инструмент для форматирования HTML-шаблонов.
pylint	Статический анализатор, к которому подключается пользовательское правило.
/tester/tests	Каталог тестовых скриптов, монтируемый из репозитория.
/tester/logs	Каталог для сохранения stdout.log и stderr.log.
python3 -m http.server 3000	Стандартный Python HTTP-сервер, запущенный как основной процесс tester.

Фрагмент установки тестовых зависимостей:

```
RUN pip3 install --no-cache-dir \
    requests==2.32.3 \
    css-html-prettify==2.5.5 \
    pylint==3.0.3

CMD service ssh start && cd /tester && exec python3 -m http.server 3000
```

Запуск стандартного HTTP-сервера в каталоге /tester соответствует требованиям ручной проверки: контейнер tester остается активным, а тестовые скрипты можно запускать отдельной командой docker compose exec tester python3 tests/run_tests.py.

2 ОПИСАНИЕ СКРИПТОВ ЗАПУСКА ТЕСТОВ

Тестовая часть проекта размещена в директории tester/tests. Запуск организован через единый pipeline run_tests.py, который последовательно выполняет проверку HTML, статический анализ и интеграционный тест.

Результат ручного запуска тестов внутри контейнера tester с использованием команды docker compose exec представлен в Приложении Б.

Команда запуска тестов:

```
docker compose exec tester python3 tests/run_tests.py
```

2.1. Общий pipeline тестирования

Скрипт run_tests.py задает базовую директорию /tester, создает каталог logs и очищает файлы stdout.log и stderr.log перед каждым запуском. Каждый этап запускается через subprocess.run с захватом stdout и stderr.

Для каждой записи формируется timestamp в формате YYYY-MM-DD HH:MM:SS. Сообщения stdout выводятся в стандартный поток, записываются в tester/logs/stdout.log и дополнительно направляются в /proc/1/fd/1, чтобы быть видимыми через docker logs. Сообщения stderr аналогично записываются в tester/logs/stderr.log и /proc/1/fd/2 (Таблица 2.1).

Таблица 2.1 – Этапы тестового pipeline

Этап	Команда	Назначение
HTML formatting test	python3 tests/test_html_format.py	Проверка HTML-шаблонов через css-html-prettify.
Custom rule test	python3 tests/test_custom_rule.py	Статический анализ по пользовательскому правилу pylint.
Upload test	python3 tests/test_upload.py	Интеграционный тест загрузки файла в веб-приложение.

2.2. Проверка HTML-шаблонов

Скрипт test_html_format.py работает с директорией /app/templates. Сначала проверяется наличие каталога шаблонов и HTML-файлов. Затем шаблоны копируются во временную директорию, после чего к копии применяется css-html-prettify.

Использование временной директории выбрано для того, чтобы тест не изменял исходники приложения. При успешном завершении утилиты скрипт выводит количество обработанных HTML-файлов.

2.3. Пользовательское правило pylint

Скрипт test_custom_rule.py реализует статический анализ через pylint. Временный файл pylint_elina_checker.py содержит пользовательский checker ElinaVariableChecker, который наследуется от BaseChecker и регистрирует сообщение forbidden-elina-variable.

Правило проверяет имена переменных, появляющиеся в узлах assignname. Если переменная имеет имя elina, checker добавляет диагностическое сообщение. При наличии такого сообщения тест завершается с ошибкой. Для исключения посторонних диагностик pylint запускается с параметрами --disable=all и --enable=forbidden-elina-variable.

Ключевая идея пользовательского правила:

```
def visit_assignname(self, node):
    if node.name == "elina":
        self.add_message("forbidden-elina-variable", node=node)
```

2.4. Интеграционный тест загрузки файла

Скрипт test_upload.py проверяет взаимодействие контейнера tester с веб-приложением app внутри Docker-сети. Для этого используется адрес http://app:5000, где app – имя сервиса из docker-compose.yml.

Тест формирует текстовый файл в памяти и отправляет POST-запрос на endpoint /upload. Успешным считается ответ с HTTP-кодом 200, корректным JSON и наличием имени отправленного файла в поле files. Такая проверка подтверждает не только доступность приложения, но и работу бизнес-сценария загрузки файла.

3 ОПИСАНИЕ DOCKER-COMPOSE КОНФИГУРАЦИИ

Файл docker-compose.yml описывает два сервиса: app и tester. Сервисы собираются из Dockerfile_app и Dockerfile_tester соответственно. Для передачи параметров используется env_file .env, а значения SSH_USER и SSH_PASSWORD также передаются в build args, поскольку пользователь и пароль создаются на этапе сборки образов.

Основные параметры сервисов docker-compose.yml приведены в таблице 3.1.

Таблица 3.1 – Сервисы docker-compose.yml

Сервис	Основные параметры	Назначение
app	build: Dockerfile_app; ports: APP_PORT:5000 и APP_SSH_PORT:22; volume app_source:/app; cpus: CPUS	Запуск Flask-приложения и предоставление SSH-доступа.
tester	build: Dockerfile_tester; depends_on: app; ports: TESTER_SSH_PORT:22; volumes app_source:/app:ro и ./tester:/tester; cpus: CPUS	Запуск тестовой среды и выполнение проверок приложения.

Сервис app публикует веб-приложение на localhost через переменную APP_PORT. Внутри контейнерной сети приложение доступно сервису tester по адресу http://app:5000. SSH-порты также проброшены только на 127.0.0.1, что ограничивает внешний доступ рамками локальной машины.

Для обмена исходниками используется именованный volume app_source. В контейнере app он монтируется как /app, а в tester — как /app:ro. Режим read-only в tester предотвращает изменение исходников приложения тестовым окружением.

Директория ./tester монтируется в контейнер tester как /tester. Благодаря этому тестовые скрипты и файлы логов доступны как внутри контейнера, так и на стороне проекта.

Переменные окружения, используемые в проекте, перечислены в таблице 3.2.

Таблица 3.2 – Переменные окружения проекта

Переменная	Значение по умолчанию	Назначение
APP_PORT	8000	Порт веб-приложения на хосте.
APP_SSH_PORT	2222	SSH-порт контейнера app на хосте.
TESTER_SSH_PORT	2223	SSH-порт контейнера tester на хосте.
SSH_USER	devops	Имя пользователя для SSH-доступа.
SSH_PASSWORD	devops80	Пароль пользователя для SSH-доступа.
CPUS	1	Ограничение числа CPU для каждого сервиса.

Фрагмент docker-compose.yml:

```
services:
  app:
    ports:
      - "127.0.0.1:${APP_PORT}:5000"
      - "127.0.0.1:${APP_SSH_PORT}:22"
    volumes:
      - app_source:/app
    cpus: "${CPUS}"


  tester:
    depends_on:
      - app
    volumes:
      - app_source:/app:ro
      - ./tester:/tester
    cpus: "${CPUS}"
```

Ограничение CPU задано через параметр cpus. Для варианта 80 расчет по формуле 1 + номер_варианта % 2 дает значение 1, поэтому в .env.example указано CPUS=1.

ЗАКЛЮЧЕНИЕ

В ходе работы была разработана Docker-инфраструктура для запуска веб-приложения и автоматизированного тестирования. Проект включает два контейнера, связанные через Docker Compose: app запускает Flask-приложение, tester выполняет проверки и обращается к приложению по внутреннему сетевому адресу http://app:5000.

Реализованы требования варианта 80: проверка HTML-шаблонов через csshtml-prettify, статический анализ с пользовательским правилом pylint, интеграционный тест загрузки файла, SSH-доступ к обоим контейнерам по паролю, логирование результатов в docker logs и отдельные файлы stdout/stderr, передача порта веб-сервера через .env и ограничение CPU.

Использование отдельных контейнеров для приложения и тестов делает конфигурацию более воспроизводимой и удобной для проверки. Тестовый pipeline позволяет одной командой выполнить все этапы контроля и получить итоговую строку ALL TESTS PASSED при успешном завершении.

Таким образом, поставленная задача выполнена: проект содержит необходимые Dockerfile, docker-compose конфигурацию, тестовые скрипты, логирование и документацию для запуска.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Docker Docs: официальная документация [Электронный ресурс]. – Текст: электронный. – URL: https://docs.docker.com/ (дата обращения: 30.04.2026).

2. Docker Compose overview: официальная документация [Электронный ресурс]. – Текст: электронный. – URL: https://docs.docker.com/compose/ (дата обращения: 30.04.2026).

3. Python 3 Documentation [Электронный ресурс]. – Текст: электронный. – URL: https://docs.python.org/3/ (дата обращения: 30.04.2026).

4. Flask Documentation [Электронный ресурс]. – Текст: электронный. – URL: https://flask.palletsprojects.com/ (дата обращения: 30.04.2026).

5. Pylint User Manual [Электронный ресурс]. – Текст: электронный. – URL: https://pylint.pycqa.org/ (дата обращения: 30.04.2026).

6. Requests: HTTP for Humans [Электронный ресурс]. – Текст: электронный. – URL: https://requests.readthedocs.io/ (дата обращения: 30.04.2026).

7. css-html-prettify 2.5.5: страница проекта PyPI [Электронный ресурс]. – Текст: электронный. – URL: https://pypi.org/project/css-html-prettify/ (дата обращения: 30.04.2026).

8. moevm/devops-examples: репозиторий GitHub [Электронный ресурс]. – Текст: электронный. – URL: https://github.com/moevm/devops-examples (дата обращения: 30.04.2026).

ПРИЛОЖЕНИЕ А. Исходный код проекта

Файл docker-compose.yml:

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile_app
      args:
        SSH_USER: ${SSH_USER}
        SSH_PASSWORD: ${SSH_PASSWORD}
    env_file:
      - .env
    ports:
      - "127.0.0.1:${APP_PORT}:5000"
      - "127.0.0.1:${APP_SSH_PORT}:22"
    volumes:
      - app_source:/app
    cpus: "${CPUS}"

  tester:
    build:
      context: .
      dockerfile: Dockerfile_tester
      args:
        SSH_USER: ${SSH_USER}
        SSH_PASSWORD: ${SSH_PASSWORD}
    env_file:
      - .env
    depends_on:
      - app
    ports:
      - "127.0.0.1:${TESTER_SSH_PORT}:22"
    volumes:
      - app_source:/app:ro
      - ./tester:/tester
    cpus: "${CPUS}"

volumes:
  app_source:
```

Файл Dockerfile_app:

```
FROM ubuntu:22.04


ARG SSH_USER=devops
ARG SSH_PASSWORD=devops80


ENV DEBIAN_FRONTEND=noninteractive
ENV APP_DIR=/app


RUN apt-get update && apt-get install -y --no-install-recommends \
    git=1:2.34.1-1ubuntu1.17 \
    python3=3.10.6-1~22.04.1 \
    python3-pip=22.0.2+dfsg-1ubuntu0.7 \
    openssh-server=1:8.9p1-3ubuntu0.14 \
```

```
    patch=2.7.6-7build2 \
    ca-certificates=20240203~22.04.1 \
    && rm -rf /var/lib/apt/lists/*


RUN mkdir -p /var/run/sshd \
    && useradd -m -s /bin/bash "${SSH_USER}" \
    && echo "${SSH_USER}:${SSH_PASSWORD}" | chpasswd \
    && sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/' /etc/ssh/sshd_config
\
    && sed -i 's/PasswordAuthentication no/PasswordAuthentication yes/' /etc/ssh/sshd_config


RUN git clone https://github.com/moevm/devops-examples.git /tmp/devops-examples \
    && mkdir -p "${APP_DIR}" \
    && cp -r /tmp/devops-examples/EXAMPLE_APP/* "${APP_DIR}" \
    && rm -rf /tmp/devops-examples


RUN sed -i "s/app.run(debug = True)/app.run(host='0.0.0.0', port=5000, debug=True)/"
"${APP_DIR}/main.py"


WORKDIR ${APP_DIR}


RUN pip3 install --no-cache-dir \
    flask==3.0.3 \
    werkzeug==3.0.3 \
    flask-login==0.6.3 \
    celery==5.3.6 \
    lti==0.9.5


CMD service ssh start && exec python3 main.py
```

Файл Dockerfile_tester:

```
FROM ubuntu:22.04


ARG SSH_USER=devops
ARG SSH_PASSWORD=devops80


ENV DEBIAN_FRONTEND=noninteractive
ENV TESTER_DIR=/tester


RUN apt-get update && apt-get install -y --no-install-recommends \
    python3=3.10.6-1~22.04.1 \
    python3-pip=22.0.2+dfsg-1ubuntu0.7 \
    openssh-server=1:8.9p1-3ubuntu0.14 \
    ca-certificates=20240203~22.04.1 \
    && rm -rf /var/lib/apt/lists/*


RUN mkdir -p /var/run/sshd \
    && useradd -m -s /bin/bash "${SSH_USER}" \
    && echo "${SSH_USER}:${SSH_PASSWORD}" | chpasswd \
    && sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/' /etc/ssh/sshd_config
\
    && sed -i 's/PasswordAuthentication no/PasswordAuthentication yes/' /etc/ssh/sshd_config


WORKDIR ${TESTER_DIR}
```

```
RUN mkdir -p ${TESTER_DIR}/tests ${TESTER_DIR}/logs


RUN pip3 install --no-cache-dir \
    requests==2.32.3 \
    css-html-prettify==2.5.5 \
    pylint==3.0.3


CMD service ssh start && cd /tester && exec python3 -m http.server 3000
```

Файл .env.example:

```
APP_PORT=8000
APP_SSH_PORT=2222
TESTER_SSH_PORT=2223


SSH_USER=devops
SSH_PASSWORD=devops80


CPUS=1
```

Файл tester/tests/run_tests.py:

```
import subprocess
import sys
from datetime import datetime
from pathlib import Path



BASE_DIR = Path(__file__).resolve().parent.parent
LOG_DIR = BASE_DIR / "logs"
STDOUT_LOG = LOG_DIR / "stdout.log"
STDERR_LOG = LOG_DIR / "stderr.log"



def timestamp():
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")



def write_stdout(message):
    line = f"[{timestamp()}] {message}"
    print(line)


    with STDOUT_LOG.open("a", encoding="utf-8") as file:
        file.write(line + "\n")


    try:
        with open("/proc/1/fd/1", "a") as docker_out:
            docker_out.write(line + "\n")
    except Exception:
        pass



def write_stderr(message):
```

```
    line = f"[{timestamp()}] {message}"
    print(line, file=sys.stderr)


    with STDERR_LOG.open("a", encoding="utf-8") as file:
        file.write(line + "\n")


    try:
        with open("/proc/1/fd/2", "a") as docker_err:
            docker_err.write(line + "\n")
    except Exception:
        pass



def run_step(name, command):
    write_stdout(f"START: {name}")


    result = subprocess.run(
        command,
        shell=True,
        cwd=BASE_DIR,
        text=True,
        capture_output=True,
    )


    if result.stdout:
        for line in result.stdout.splitlines():
            write_stdout(f"{name}: {line}")


    if result.stderr:
        for line in result.stderr.splitlines():
            write_stderr(f"{name}: {line}")


    if result.returncode == 0:
        write_stdout(f"SUCCESS: {name}")
        return True


    write_stderr(f"FAIL: {name} with exit code {result.returncode}")
    return False



def main():
    LOG_DIR.mkdir(parents=True, exist_ok=True)


    STDOUT_LOG.write_text("", encoding="utf-8")
    STDERR_LOG.write_text("", encoding="utf-8")


    steps = [
        ("HTML formatting test", "python3 tests/test_html_format.py"),
        ("Custom rule test", "python3 tests/test_custom_rule.py"),
        ("Upload test", "python3 tests/test_upload.py"),
    ]


    all_passed = True


    for name, command in steps:
        if not run_step(name, command):
```

```
            all_passed = False

    if all_passed:
        write_stdout("ALL TESTS PASSED")
        sys.exit(0)

    write_stderr("SOME TESTS FAILED")
    sys.exit(1)



if __name__ == "__main__":
    main()
```

Файл tester/tests/test_html_format.py:

```
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path



APP_DIR = Path("/app")
SOURCE_TEMPLATES_DIR = APP_DIR / "templates"



def fail(message):
    print(message, file=sys.stderr)
    sys.exit(1)



def main():
    if not SOURCE_TEMPLATES_DIR.exists():
        fail(f"Templates directory not found: {SOURCE_TEMPLATES_DIR}")

    html_files = list(SOURCE_TEMPLATES_DIR.glob("*.html"))

    if not html_files:
        fail("No HTML files found for formatting")

    with tempfile.TemporaryDirectory() as temp_dir:
        temp_templates_dir = Path(temp_dir) / "templates"
        shutil.copytree(SOURCE_TEMPLATES_DIR, temp_templates_dir)

        result = subprocess.run(
            [
                "python3",
                "/usr/local/bin/css-html-prettify.py",
                str(temp_templates_dir),
            ],
            text=True,
            capture_output=True,
        )
```

```
        print(result.stdout)

        if result.stderr:
            print(result.stderr, file=sys.stderr)

        if result.returncode != 0:
            fail("HTML formatting failed")

    print(f"HTML formatting passed for {len(html_files)} files")



if __name__ == "__main__":
    main()
```

Файл tester/tests/test_custom_rule.py:

```
import subprocess
import sys
import tempfile
from pathlib import Path



APP_DIR = "/app"



CHECKER_CODE = """
from pylint.checkers import BaseChecker



class ElinaVariableChecker(BaseChecker):
    name = "elina-variable-checker"

    msgs = {
        "C9001": (
            "Forbidden variable name 'elina'",
            "forbidden-elina-variable",
            "Used when a variable has the forbidden name 'elina'.",
        ),
    }

    def visit_assignname(self, node):
        if node.name == "elina":
            self.add_message("forbidden-elina-variable", node=node)



def register(linter):
    linter.register_checker(ElinaVariableChecker(linter))
"""



def fail(message):
    print(message, file=sys.stderr)
    sys.exit(1)
```

```
def main():
    with tempfile.TemporaryDirectory() as temp_dir:
        checker_path = Path(temp_dir) / "pylint_elina_checker.py"
        checker_path.write_text(CHECKER_CODE, encoding="utf-8")

        result = subprocess.run(
            [
                "pylint",
                f"--init-hook=import sys; sys.path.insert(0, '{temp_dir}')",
                "--load-plugins=pylint_elina_checker",
                "--disable=all",
                "--enable=forbidden-elina-variable",
                APP_DIR,
            ],
            text=True,
            capture_output=True,
        )

    if result.stdout:
        print(result.stdout)

    if result.stderr:
        print(result.stderr, file=sys.stderr)

    if "forbidden-elina-variable" in result.stdout:
        fail("Custom pylint rule failed: forbidden variable name 'elina' was found")

    if result.returncode not in (0, 16):
        fail(f"pylint finished with unexpected return code {result.returncode}")

    print("Custom pylint rule passed: variable name 'elina' was not found")



if __name__ == "__main__":
    main()
```

Файл tester/tests/test_upload.py:

```
import sys
from io import BytesIO

import requests



APP_URL = "http://app:5000"
UPLOAD_URL = f"{APP_URL}/upload"
TEST_FILENAME = "test_upload_file.txt"
TEST_CONTENT = b"DevOps upload test content"



def fail(message):
    print(message, file=sys.stderr)
```

```
    sys.exit(1)



def main():
    files = {
        "file": (TEST_FILENAME, BytesIO(TEST_CONTENT), "text/plain")
    }

    response = requests.post(UPLOAD_URL, files=files, timeout=10)

    if response.status_code != 200:
        fail(f"Expected status code 200, got {response.status_code}")

    try:
        response_data = response.json()
    except ValueError:
        fail("Upload response is not valid JSON")

    uploaded_files = response_data.get("files", [])

    if TEST_FILENAME not in uploaded_files:
        fail(
            f"Uploaded file '{TEST_FILENAME}' was not found in response: "
            f"{uploaded_files}"
        )

    print(f"Upload integration test passed: {TEST_FILENAME}")



if __name__ == "__main__":
    main()
```

ПРИЛОЖЕНИЕ Б. Ручной запуск тестов в контейнере tester

vboxuser@Ubuntu:~/Документы/DevOps_project$ docker compose exec tester python3 tests/run_tests.py

[2026-04-29 17:27:11] START: HTML formatting test

[2026-04-29 17:27:12] HTML formatting test: CSS-HTML-Prettify.

[2026-04-29 17:27:12] HTML formatting test:

[2026-04-29 17:27:12] HTML formatting test: StandAlone Async cross-platform Prettifier Beautifier for the Web.

[2026-04-29 17:27:12] HTML formatting test: 2.5.5

[2026-04-29 17:27:12] HTML formatting test: Target is a Folder with CSS / SCSS, HTML, JS.

[2026-04-29 17:27:12] HTML formatting test: Processing a whole Folder may take some time...

[2026-04-29 17:27:12] HTML formatting test: Process 26 is processing /tmp/tmpllu1x2w_/templates/index.html.

[2026-04-29 17:27:12] HTML formatting test: Monkey Patching BeautifulSoup on-the-fly to process HTML...

[2026-04-29 17:27:12] HTML formatting test: Process 25 is processing /tmp/tmpllu1x2w_/templates/files.html.

[2026-04-29 17:27:12] HTML formatting test: Monkey Patching BeautifulSoup on-the-fly to process HTML...

[2026-04-29 17:27:12] HTML formatting test: Process 25 is processing /tmp/tmpllu1x2w_/templates/upload.html.

[2026-04-29 17:27:12] HTML formatting test: Monkey Patching BeautifulSoup on-the-fly to process HTML...

[2026-04-29 17:27:12] HTML formatting test:

[2026-04-29 17:27:12] HTML formatting test: ----------

[2026-04-29 17:27:12] HTML formatting test: Files Processed: ['/tmp/tmpllu1x2w_/templates/files.html', '/tmp/tmpllu1x2w_/templates/index.html', '/tmp/tmpllu1x2w_/templates/upload.html'].

[2026-04-29 17:27:12] HTML formatting test: Number of Files Processed:

[2026-04-29 17:27:12] HTML formatting test: 1

[2026-04-29 17:27:12] HTML formatting test:

[2026-04-29 17:27:12] HTML formatting test: HTML formatting passed for 3 files

[2026-04-29 17:27:12] HTML formatting test: /usr/local/bin/css-html-prettify.py:347: GuessedAtParserWarning: No parser was explicitly specified, so I'm using the best available HTML parser for this system ("html.parser"). This usually isn't a problem, but if you run this code on another system, or in a different virtual environment, it may use a different parser and behave differently.

[2026-04-29 17:27:12] HTML formatting test:

[2026-04-29 17:27:12] HTML formatting test: The code that caused this warning is on line 347 of the file /usr/local/bin/css-html-prettify.py. To get rid of this warning, pass the additional argument 'features="html.parser"' to the BeautifulSoup constructor.

[2026-04-29 17:27:12] HTML formatting test:

[2026-04-29 17:27:12] HTML formatting test: html = BeautifulSoup(html).prettify()

[2026-04-29 17:27:12] HTML formatting test: /usr/local/bin/css-html-prettify.py:347: GuessedAtParserWarning: No parser was explicitly specified, so I'm using the best available HTML parser for this system ("html.parser"). This usually isn't a problem, but if you run this code on another system, or in a different virtual environment, it may use a different parser and behave differently.

[2026-04-29 17:27:12] HTML formatting test:

[2026-04-29 17:27:12] HTML formatting test: The code that caused this warning is on line 347 of the file /usr/local/bin/css-html-prettify.py. To get rid of this warning, pass the additional argument 'features="html.parser"' to the BeautifulSoup constructor.

[2026-04-29 17:27:12] HTML formatting test:

[2026-04-29 17:27:12] HTML formatting test: html = BeautifulSoup(html).prettify()

[2026-04-29 17:27:12] HTML formatting test:

[2026-04-29 17:27:12] SUCCESS: HTML formatting test

[2026-04-29 17:27:12] START: Custom rule test

[2026-04-29 17:27:14] Custom rule test: Custom pylint rule passed: variable name 'elina' was not found

[2026-04-29 17:27:14] SUCCESS: Custom rule test

[2026-04-29 17:27:14] START: Upload test

[2026-04-29 17:27:15] Upload test: Upload integration test passed: test_upload_file.txt

[2026-04-29 17:27:15] SUCCESS: Upload test

[2026-04-29 17:27:15] ALL TESTS PASSED